Temco Platform

The editor · built Nov 12, 2026

The editor

The design for **R6 — editing and publishing**: the toolbar, creating a document, images, drafts, and where the review happens. It is written the same week its first step was built, so it separates what exists from what is proposed, and it puts the one decision that is not mine in front of the person who has to make it.

The content rules are not here. Registry schema, link forms, media rules, review flow and publishing belong to `design/temco-learn/`, which stays the source of truth; this document only says how the desktop application implements them.

1. The surface: VS Code's editor, with the project's toolbar on it

`15-online-editor.md` designs the **hosted editor**: Monaco, bundled and self-hosted, with an API behind it, served by the center tool on a host that has a clone. That editor is not this application's editor and building it twice would be the bug.

	the hosted editor	this application
the core	Monaco, bundled by us, ~2 MB, custom build, `<textarea>` fallback	**VS Code is Monaco plus a shell** — nothing to bundle
what it adds	toolbar, preview pane, gate markers, upload, publish, PR	toolbar, a link to a real page, the outline, the gates' markers — of which the markers already exist (`src/diagnostics.ts`)
what it never does	never WYSIWYG: a rich round-trip rewrites the line layout of the whole file	the same rule, and the same reason
publishing	`POST /api/docs/publish` → a real PR	**never.** This application builds and reports; shipping stays a separate act

So the desktop equivalent of "Monaco + the toolbar + completion + markers" is **VS Code, with the toolbar built** — which is what `src/format.ts` now is. An editor page of the project's own, hosting Monaco in a webview, was considered and is refused for the reason `15-online-editor.md`, *What is not*

built gives for the website: the integration is the part that matters, and the editor is the part that already exists.

What exists — the toolbar, built

Sixteen commands, all `temcoDocs.format.*`, in `src/format.ts`. Each one edits the buffer in front: the page a person is typing in, undone with `Ctrl+Z`. None of them writes a file, opens a tab, or touches the registry.

Where a person meets it	What is there
the editor's title bar, Markdown files only	**Bold** · *Italic* · *Link to a page* · **Format**
the editor's context menu, and the tab's	the submenu **Temco Docs** — all sixteen, grouped
the palette	every action, by name; **Format** lists them all in one picker

Group	Actions
Inline	Bold `**…**` · Italic `*…*` · Inline code `` `…` ``
Blocks	Heading level (`##` – `####`) · bullet · numbered · task · block quote · code block with a language · table · horizontal rule · callout
Insert	Link · **Link to a page**
Navigate	Go to a heading (this page's outline)

Three things it does on purpose, each of which has an obvious wrong answer:

- **Every button is a toggle.** Bold over `**bold**` removes the marks; the bullet list over a list takes the bullets off; the same heading level again removes it. That is what a button does in every other editor.
- **A block gets a blank line before it when the line above is prose.** Not decoration: `text` followed by `---` is a level-2 heading in Markdown, and a table glued to a paragraph is a paragraph.
- **A callout, a fence and an alert marker start their own line** — mid-line they would glue to the text after the cursor, and `> [!NOTE]` is an alert only at the start of a line.

No keybindings are contributed, deliberately. `Ctrl+B` hides the side bar everywhere else, and this extension has to keep working in an ordinary VS Code. Everything here is a command, so a chord is one personal binding away.

A page that is not a file on disk says so. A page read from a source, or a diff, has a scheme that is not `file`, and every action refuses it in words rather than appearing to work.

---

2. Creating a document

A page is a file, not a pair — and that is the product repository's own doing. This section said the opposite when it was first written, so the correction is worth stating plainly, with where it comes from. `T3000Webview-EEZ-Studio/scripts/docs-registry.mjs` **generates the entries**:

```
TEXT

  buildPages(existing)                    walks docs/t3000/** and docs/legacy/**
    id      = the docs-relative path without `.md`      the page id *is* the path
    title   = prev.title  ??  the file's first `#`      ← so a skeleton starts with one
    area    = the top folder      chapter = the second path segment (or `(root)`)
    order   = prev.order  ??  10, 20, 30… inside the chapter
    owner   = 'docs'   ship = true for t3000   tags/keywords/code = []
    every field:  prev.<field> ?? a default             ← an existing (curated) value al
  and the file itself is written with  JSON.stringify(registry, null, 2) + '\n'
```

So **creating a page writes no entry at all**: the file goes in, the repository's own generator — which this application already runs as one of its allowlisted scripts — adds the entry, derives its place, and takes its title from the first heading. What only a person curates is `order`, `chapter`, `title`, `owner`, `tags` and `code`, and those are the values the generator then leaves alone. Section 5 is that decision, as taken.

Two consequences for the flow below, both now built: a new page is **a file and a generator run**, offered in that order; and there is **no "loose page" kind**, because the `exclude` key the design package's schema describes is not produced by this repository's generator — a Markdown file under `docs/` that no area covers is an `unregistered page` error, not a working note.

The kinds worth offering

Kind	The headings it starts with	Why it is a kind
Article	Overview · Configuration · Related pages	the ordinary case
Task	Before you start · Steps · If it does not work	the commonest documentation shape
Reference	Fields · Notes	the shape an API or a register actually has
Troubleshooting	Symptom · Cause · Fix	matches the `troubleshooting` area the product already has
Release note	What is new · Fixed · Known issues	what changed in a build

Each skeleton is **headings only** — no invented sentences, because a template that writes prose is a template someone has to delete. Every one starts with `# <title>`, which is what the generator reads.

The flow, as built

```text
New page ──────────────────────────────────────────────────────────────────────
1  Where    the row it was started from — a page's own area and chapter, a chapter its
            an area's root chapter — or the row you have selected, or:
            which source ▸ which area ▸ which chapter
2  Kind     article · task · reference · troubleshooting · release note
3  Title    Trend Logs
4  File     trend-logs          →  docs/t3000/features/trend-logs.md        (shown, no
──────────────────────────────────────────────────────────────────────────────

[ Write the page ]  ── it starts as “# Trend Logs” and the kind's headings
── then ──
“It becomes a page when the registry names it.”          [ Run the generator ]
```

A repository row cannot answer step 1 — it is not in an area — so it answers the source and the rest is still asked. Nothing else is asked when the kind came from the *New Page of a Kind* submenu: step 2 is the entry that was clicked, and steps 3 and 4 follow as they always do.

New chapter is the same flow with the folder named first, because a chapter is a page's *path*: a folder with nothing in it is not a chapter, not in the registry, and not in the table of contents. The folder arrives with its first page.

---

3. Images

An image is the one action in the toolbar that is missing, and the reason is not the code: the file has to be written into the page's own `media/` folder before `![alt](media/…)` points at anything.

The rules are already written down, in the design package, so they are not ours to invent:

Rule	Source
the file goes into the **page's own** `media/`, with a slugified name (`02-run-screen.png`)	`15-online-editor.md`, *Uploads*
accepted: PNG, JPG, WebP, GIF, SVG — diagrams as SVG, sanitised on the way in	`15-online-editor.md`, *Uploads*
images ≤ 2 MB · a page ≤ 1 MB · content-type sniffed, never trusted from the name	`15-online-editor.md`, *Safety rules*, rule 2
the destination is derived, never supplied	`15-online-editor.md`, *Safety rules*, rule 1
screenshots are captured properly or not at all — no editing, no resizing	`15-online-editor.md`, *What is not built*

What an extension can and cannot take. A file the user **picked** (`showOpenDialog`), a file **dropped** on the page and an image already on the clipboard are all reachable — the last one not through the extension API, which exposes the clipboard's *text* only. On Windows the module reads the bitmap itself (`System.Windows.Forms.Clipboard`, one child process per capture), which is what makes `Ctrl+Alt+V` work; the loop it belongs to, and what is still missing from it, is `authoring-flow.md`.

---

4. Drafts, review and publishing

Not built, and the largest piece of R6. The design is already fixed and is not ours to change:

	where it belongs
editing a **local** source	in place — the page file is the page, which the Workspace view already does
editing a **git** source	on a draft branch in a materialised clone — which is the `fetch` read mode R2 still owes
review	`docs:blocking`, `registry --check`, the link check and the media check, run by the repository's own scripts
publishing	never from this application: the gates, the commit and the PR are the product repository's
a page's fields in the editor	restricted by `15-online-editor.md`, *Safety rules*, rule 4: structure needs the docs owner

So the honest order is: **images and create-a-page first, because both need only a decision**; drafts after, because they need the `fetch` mode and a review flow that lives in the product repository.

5. The decision, as taken

The question was how far this application may go into a product repository. The answer, in the three parts it was asked in:

Asked	Taken	So the code does this
The Library's title bar holds twelve buttons	**three inline, the rest in the view's `…`**	`newProject` · `openProjects` · `openNeeds` stay in group `navigation`; everything else moved to named groups, which is how a `view/title` item lands in the overflow menu. **The header starts work; a row and a page operate on a thing** — and the target-less ones ask which, which they already did
Which operations the Workspace tree grows	**all five**: new page · new chapter · rename the title · move (up, down, and by dragging) · retire	`src/pageOps.ts` — six commands, a `TreeDragAndDropController`, and one modal per write
How far into `registry.json`	**one named value, on a confirmation**	`src/registryEdit.ts` — `title`, `order`, `chapter`, `deprecated`, `superseded_by`, and nothing else; never a whole-file rewrite

Where a row offers its operations was settled on the third attempt: **the row menu is the Explorer's, the toolbar mirrors it, and the hover carries nothing.** A **left click** does what it does in the Explorer — opens a page, expands a folder — and a **right click** (or the keyboard's menu key) opens *New Page…* · *New Chapter…* · *New Page of a Kind* ▸ · *Reveal in Explorer* · *Open in Integrated Terminal* · *Copy Path* · *Copy Relative Path* · *Rename the Title…* · *Move the Page Up* · *Move the Page Down* · *Retire…* · *Open the Registry*, each item only where it means something. The view's `…` shows the same list — plus *New page*, *New chapter*, *Refresh*, Home and Validate — acting on the row the tree last had focused, which is also what makes it usable from the palette: **a view title action is run with the tree pane's context**, and every row command falls back to the view's own selection. There are no hover icons on a row: a menu started from a row can only ever be a quick pick in the middle of the top of the window, and that is exactly what the row menu above replaces.

What each operation writes

TEXT

operation	what changes on disk	state
Open · Reveal · the outline	nothing	built
Create the missing page file	one empty .md (declared, absent)	built (e
New page in this chapter	one .md from a kind's skeleton	built
→ then the repo's docs:registry	(its own script; adds the entry)	offered,
New chapter	…the same, with the folder named first	built
Rename the title	title in registry.json (+ the page's first #)	built
Move up / Move down / drag	order in registry.json — two numbers, swapped	built
Drop on another chapter	chapter + order — *structure*, docs owner	built
Retire a page	deprecated + superseded_by	built
Rename or move the FILE	the page id — inbound links break	**never*
An image into the page's media/	a new file, binary	**not b

Four rules hold the writes together, and each one is a decision rather than an implementation detail:

- **A move is a swap, never a renumbering.** Placing a page between two others by rewriting every `order` in the chapter is the generator's own job — it hands out 10, 20, 30… to whatever nobody curated. So a move writes two numbers and the diff is two lines.
- **The file is never renamed or moved.** The page id *is* the path (`05-content-and-registry.md` , *Page ids and aliases*), so a page is renamed by changing what it is *called*: one string in the registry, plus the page's own `#` heading when it says the same words — two answers to one question otherwise.
- **A chapter change says the word structure.** `15-online-editor.md` , *Safety rules*, rule 4 marks `chapter` as the docs owner's to change, so the confirmation says so and asks for a line in the pull request.
- **Retirement is not deletion.** The file stays; the registry records the replacement; the redirect for a retired page is generated by the repository's own pipeline (`14-governance.md` , *Deprecation and retirement*).

The evidence for "one value, and nothing else"

`src/registryEdit.ts` renders the file the way the repository's **own generator** writes it — `JSON.stringify` with two spaces, one trailing newline — **and with the file's own line endings**, because a clone checked out on Windows is CRLF while the generator emits LF, and a write that flipped every ending would be a diff of the whole file. With both right, the guarantee is testable, so it was tested:

Check	Result
the real `docs/help/registry.json` — 318 pages, 124 KB — opened, re-rendered, compared	**byte-for-byte identical**
a drag into another chapter, diffed line by line against the original	**two lines changed**: `chapter` and `order`
a drag onto a sibling, same chapter	two lines changed, the two `order` values swapped
a drop on the page's own chapter	**nothing written**, and it says so
a rename, diffed	one line changed, plus the page's own heading
a new page	the file written, **the registry untouched**, then the generator offered

Running the generator afterwards is what keeps the file canonical: the values written here are the ones the generator's own `prev.<field> ?? default` rule preserves, so nothing is lost by it and nothing here duplicates it.

---

6. Acceptance, per step

#	Step	The test that ends it
1	the toolbar	**done** — a Markdown file shows the four buttons; every button toggles; `---` after prose is not a heading; the page picker inserts a link the gate resolves; a page read from a source is refused in words
2	operating on a page	**done** — a page is created from its kind's skeleton and the generator is offered next; a title is renamed and the registry diff is one line; a page is dragged into another chapter and the diff is the two values; retiring one writes `deprecated` and the replacement and keeps the file
3	insert an image	a dropped PNG lands in the page's `media/` under a slugified name and is referenced relatively; a 3 MB PNG and a `.exe` renamed `.png` are both refused, in words, with the limit
4	drafts	a page is corrected, the gates run, the diff is reviewed, and the result is a commit a person made — not one this application made